building up layers of options configuration, as shown previously in figure 31.5.

Listing 31.13 Configuring an `IOptions<T>` object using `OptionsBuilder<T>`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddOptions<CurrencyOptions>()                          ❶
    .BindConfiguration("Currencies")                        ❷
    .Configure(opts =>                                      ❸
        opts.Currencies = new string[] { "GBP", "USD"});    ❸

WebApplication app = builder.Build();
app.MapGet("/", (IOptions<CurrencyOptions> opts) => opts.Value);
app.Run();
```

❶ Creates an OptionsBuilder<CurrencyOptions> object

❷ Binds to the Currencies section of the IConfiguration

❸ Configures the IOptions object by executing a lambda function

You've seen the builder pattern many times throughout the book, and the pattern in this case is no different. The builder exposes methods that you can chain together fluently. One of the benefits of the builder pattern is that it's easy to discover all the methods it exposes. In this case, if you explore the type in your integrated development environment (IDE), you may notice that `OptionsBuilder<T>` exposes multiple Configure overloads, such as

- `Configure<TDep>(Action<T,TDep> config);`
- `Configure<TDep1,TDep2>(Action<T, TDep1, TDep2> config);`
- `Configure<TDep1,TDep2,TDep3> (Action<T,TDep1,TDep2,TDep3> config);`